

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

дисциплина: Математические основы защиты информации и
информационной безопасности

Студент: Пиняева Анна Андреевна

Группа: НПИмд-01-24

МОСКВА

2025

Теоретическое введение

Алгоритм Полларда — это эффективный метод для нахождения дискретного логарифма, который использует идею случайных блужданий и метод “кролика и черепахи”.

Определение задачи

Дискретный логарифм — это задача нахождения целого числа x по заданным элементам g и y в конечной группе G , такой что:

$$g^x \equiv y \pmod{p}$$

где g — основание, y — результат возведения в степень, а p — простое число, определяющее порядок группы.

Основная идея алгоритма

Алгоритм Полларда основан на методе случайных блужданий и использует принцип “кролика и черепахи” (или “метод Флойда”). Он предполагает, что мы можем генерировать последовательности значений с помощью случайных блужданий и сравнивать их для нахождения совпадений.

Алгоритм Полларда является мощным инструментом для решения задачи дискретного логарифмирования в конечных группах. Его эффективность в основном зависит от выбора начальных значений и структуры группы. Этот алгоритм часто используется в криптографии, особенно в контексте систем на основе эллиптических кривых и других методов шифрования.

Цель работы

Ознакомиться с алгоритмом дискретного логарифмирования в конечном поле.

Ход работы

Реализовать алгоритм дискретного логарифмирования в конечном поле

```
function searching_for_gamma(a_diff, b_diff, p)
    for i in 1:p
        if b_diff*i %p == a_diff
            return i
        end
    end
    return "Not found"
end
function new_xab(x, a, b, p, alph, bett)
    if x % 3 == 0
        return x^2 % p, a*2 % (p-1), b*2 % (p-1)
    elseif x % 3 == 1
        return x*alph % p, (a+1) % (p-1), b
    else
        return x*bett % p, a, (b+1) % (p-1)
    end
end
function metodPollarda(p, alp, bet)# , any_func::Function)
    if p % 2 == 0
        return "Incorrect input: p must be simple"
    end
    a_i = 0; b_i = 0; x_i = 1
    a_2i = 0; b_2i = 0; x_2i = 1
    i = 1
    tries = 1000
    data = zeros(Int64, (3, tries))
    data2 = zeros(Int64, (3, tries))
    while i <= tries
        x_i, a_i, b_i = new_xab(x_i, a_i, b_i, p, alp, bet)
        data[:, i] = [x_i, a_i, b_i]

        x_2i, a_2i, b_2i = new_xab(x_2i, a_2i, b_2i, p, alp, bet)
        x_2i, a_2i, b_2i = new_xab(x_2i, a_2i, b_2i, p, alp, bet)
        data2[:, i] = [x_2i, a_2i, b_2i]

        if x_i == x_2i
            display(data[:, 1:i])
            display(data2[:, 1:i])
            r = b_2i - b_i
            if r == 0
                return "Не найдено"
            end
        end
    end
end
```

```

        else
            return searching_for_gamma(a_i - a_2i, r, p)
        end
    end
    i += 1
end
return "Делитель не найден"
end

```

Тестирование

```

p=107
alp=10
bet=64
metodPollarda(p,alp,bet)

```

```

p=1011
alp=7
bet=3
metodPollarda(p,alp,bet)

```

Результат тестирования представлен на рис.1

```

[5]: p=107
      alp=10
      bet=64
      metodPollarda(p,alp,bet)

3x14 Matrix{Int64}:
 10 100 37 49 62 9 81 34 19 83 69 53 75 61
 1 2 3 4 5 5 10 20 21 22 22 44 44 88
 0 0 0 0 0 1 2 4 4 4 5 10 11 22

3x14 Matrix{Int64}:
100 49 9 34 83 53 61 61 61 61 61 61 61 61
 2 4 5 20 22 44 88 72 40 82 60 16 34 70
 0 0 1 4 4 10 22 44 88 70 34 68 30 60

[5]: 23

[10]: p=1011
       alp=7
       bet=3
       metodPollarda(p,alp,bet)

3x56 Matrix{Int64}:
 7 49 343 379 631 373 589 79 ... 103 721 1003 955 619 289 1
 1 2 3 4 5 6 7 8 50 51 52 53 54 55 56
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

3x56 Matrix{Int64}:
49 379 373 79 838 622 148 175 ... 499 187 64 103 1003 619 1
 2 4 6 8 10 12 14 16 100 102 104 106 108 110 112
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

[10]: "Не найдено"

```

Рис. 1 Тестирование:

Вывод: В результате работы мы ознакомились с алгоритмом дискретного логарифмирования в конечном поле и реализовали его на языке программирования Julia.